

Factora

Funktions- & API-Referenz

E-Invoicing-Plattform · XRechnung · ZUGFeRD · Peppol · DATEV · GoBD
Vollständige technische Referenz

Inhalt

- 01 Überblick
- 02 Rechnungstypen
- 03 Rechnungsformate
- 04 Steuerlogik
- 05 Compliance & Archivierung
- 06 API-Überblick
- 07 Endpoints
- 08 Atomic Invoice — Feldschema
- 09 Webhooks
- 10 Exporte

Factora — Funktions- & API-Referenz

Vollständige technische Referenz der Factora E-Invoicing-Plattform: was die Plattform kann, welche Rechnungsformate sie erzeugt, welche Steuerlogik greift und wie die öffentliche REST-API im Detail funktioniert — inklusive aller Felder, Rechnungstypen und Endpoints.

Diese Referenz ist format- und integrationsneutral. Sie beschreibt die Plattform allgemein, nicht einen einzelnen Anwendungsfall.

Überblick

Factora ist eine E-Invoicing-Plattform für Deutschland und die EU. Sie nimmt Rechnungsdaten in einem strukturierten Format entgegen und erzeugt daraus gesetzeskonforme elektronische Rechnungen nach **EN 16931** — als **XRechnung** (CII), **ZUGFeRD** (PDF/A-3 mit eingebetteter XML) und **Peppol/UBL**. Jede finalisierte Rechnung wird gegen das amtliche Regelwerk geprüft, revisionssicher (GoBD) archiviert und kann per DATEV exportiert werden.

Es gibt zwei Wege, Factora zu nutzen:

- **Web-Anwendung** — Rechnungen im Browser erfassen, finalisieren, versenden.
- **Öffentliche REST-API** — Rechnungserstellung direkt aus einem Fremdsystem (ERP, Warenwirtschaft, Abrechnungssoftware). Ein einziger Aufruf erzeugt eine fertige, geprüfte Rechnung als PDF und XML.

Inhaltsverzeichnis

Plattform & Engine

- Rechnungstypen — alle unterstützten Rechnungsarten und ihre TypeCodes (Rechnung, Korrektur, Abschlag, kumulierte Abschläge, Schlussrechnung, Gutschrift/Self-Billing).
- Rechnungsformate — XRechnung (CII), ZUGFeRD (PDF/A-3), Peppol/UBL; Profile, Prüfung und amtliche Abnahme (KoSIT).
- Steuerlogik — alle USt-Kategorien, Steuersätze, Befreiungsgründe, §13b Reverse Charge, innergemeinschaftliche Lieferung, Ausfuhr, §19 Kleinunternehmer, grenzüberschreitende Logik, Prüfregele.
- Compliance & Archivierung — GoBD-Unveränderlichkeit, verschlüsseltes Archiv, Änderungsprotokoll, EN-16931-Validierung.

Öffentliche API

- API-Überblick — Basis-URL, Authentifizierung, API-Schlüssel, Sandbox/Live, Idempotenz, Limits, Antwort-Hülle, Fehlercodes, Scopes.
- Endpoints — vollständige Liste aller Endpunkte mit Methode, Zweck, erforderlichem Scope und Verhalten.
- Atomic Invoice — vollständiges Feldschema — jedes Feld des Ein-Aufruf-Rechnungsendpunkts (Kopf, Verkäufer, Käufer, Positionen, Summen, Anhänge) mit Typ, Pflicht/Optional, Default und EN-16931-Bezug.
- Webhooks — Ereignistypen, Signaturprüfung, Zustellung & Wiederholungslogik.
- Exporte — DATEV-Export (Buchungsdaten).

Konventionen in dieser Referenz

- **BT-/BG-Codes** verweisen auf die Geschäftsbegriffe (Business Terms / Groups) der Norm **EN 16931**.
- **TypeCodes** (z. B. 380, 384) stammen aus **UNTDID 1001**, **Steuerkategorien** aus **UNTDID 5305**, **Maßeinheiten** aus **UN/ECE Rec. 20 / UNCL 5272**.
- Geldbeträge werden als Dezimal-Strings mit 2 Nachkommastellen übertragen (z. B. "**1190.00**"), Mengen mit 3 (z. B. "**10.500**"), Sätze mit 2 (z. B. "**19.00**"). Datumsangaben sind **ISO 8601** (**YYYY-MM-DD**).

Interaktive Doku

Die maschinenlesbare OpenAPI-3.1-Spezifikation mit Ausprobier-Funktion ist öffentlich erreichbar unter app.factora.software/api/docs/.

Rechnungstypen

Factora unterstützt die in Deutschland und der EU gebräuchlichen Rechnungsarten. Der Rechnungstyp wird über den **TypeCode (BT-3, UNTDID 1001)** gesteuert und bestimmt die rechtliche Einordnung sowie die Struktur des erzeugten Dokuments.

Unterstützte TypeCodes

TypeCode	Rechnungsart	Bedeutung
380	Rechnung	Standard-Rechnung (Voreinstellung)
384	Korrekturrechnung	Berichtigung einer bereits gestellten Rechnung
386	Abschlagsrechnung	Voraus-/Teilzahlung (auch kumulierte Abschläge)
389	Gutschrift (Gutschriftverfahren / Self-Billing)	Der Leistungsempfänger stellt die Rechnung im Namen des Leistenden aus (§14 Abs. 2 UStG)

Über die öffentliche API wird der Typ im Feld `invoice_header.invoice_type` gesetzt — entweder als TypeCode ("**380**") oder als semantisches Kürzel. Ohne Angabe gilt **380** (Standard-Rechnung).

Die Rechnungsarten im Detail

Standard-Rechnung (380)

Die normale Ausgangsrechnung. Enthält Verkäufer, Käufer, Positionen, Steueraufstellung und Zahlungsangaben. Erzeugt XRechnung-XML und ZUGFeRD-PDF.

Abschlagsrechnung (386)

Für Voraus- oder Teilzahlungen vor vollständiger Leistungserbringung — typisch bei längeren Projekten.

- **Einzelne Abschläge** werden als Abschlagsrechnung mit TypeCode 386 ausgewiesen.
- **Kumulierte Abschläge** werden unterstützt: bereits gestellte Abschläge werden fortlaufend aufsummiert geführt, sodass die Folgerechnung den korrekten kumulierten Stand trägt. Das deckt das im Bauwesen übliche Vorgehen nach **VOB/B §16** ab.

Schlussrechnung

Schließt ein Projekt ab und verrechnet die zuvor geleisteten Abschläge gegen den Gesamtbetrag. Die Schlussrechnung wird als reguläre Rechnung (TypeCode 380) geführt; der Bezug zu den Abschlägen wird über die kumulierte Abschlagsführung hergestellt.

Korrekturrechnung (384)

Berichtigt eine bereits gestellte Rechnung. Über das Feld `invoice_header.parent_invoice` wird die ursprüngliche Rechnung referenziert (BG-3 / BT-25 / BT-26). Im UBL/Peppol-Format wird eine Korrektur als **CreditNote** abgebildet.

Gutschrift / Self-Billing (389)

Das **Gutschriftverfahren** nach §14 Abs. 2 UStG: nicht der Leistende, sondern der **Leistungsempfänger** stellt die Rechnung aus (und vergibt damit auch die Rechnungsnummer). Anwendungsfall z. B. Abrechnung durch den Auftraggeber.

Hinweis zum Reifegrad: Im XRechnung/CII-Pfad ist der TypeCode 389 erzeugbar und amtlich (KoSIT) geprüft. Im internationalen Peppol/UBL-Pfad wird das eigenständige **Peppol-Self-Billing-Profil** erzeugt; dessen amtliche Abnahme über einen Peppol-Self-Billing-Validator steht noch aus (siehe Rechnungsformate).

Abgrenzung: Begriff „Gutschrift“

In der Praxis wird „Gutschrift“ doppeldeutig verwendet:

- **Kaufmännische Gutschrift / Stornierung** einer Rechnung → das ist eine **Korrekturrechnung (384)**.
- **Gutschrift im umsatzsteuerlichen Sinn (Self-Billing)** → das ist der **TypeCode 389**.

Factora trennt beide Fälle sauber über den TypeCode.

Rechnungsformate

Factora erzeugt elektronische Rechnungen nach der europäischen Norm **EN 16931**. Aus denselben Rechnungsdaten werden je nach Bedarf verschiedene Ausgabeformate generiert.

Unterstützte Formate

Format	Syntax	Datei	Einsatzzweck
XRechnung	CII (UN/CEFACT)	XML	Amtliches deutsches B2B-/B2G-Format
ZUGFeRD	CII, eingebettet	PDF/A-3	Menschen- und maschinenlesbare Hybrid-Rechnung
Peppol / UBL	UBL (OASIS)	XML	Internationaler Versand über das Peppol-Netzwerk
UBL CreditNote	UBL	XML	Korrekturrechnung im Peppol-Format

XRechnung (CII)

Die **XRechnung** ist der deutsche CIUS (Core Invoice Usage Specification) der EN 16931 und Pflichtformat für Rechnungen an öffentliche Auftraggeber (B2G). Factora erzeugt die XRechnung in **CII-Syntax** (UN/CEFACT Cross-Industry Invoice). Unterstützt werden u. a. Leitweg-ID (B2G), Reverse-Charge-Hinweise, Abschläge und Korrekturbezüge.

ZUGFeRD (PDF/A-3)

ZUGFeRD ist eine Hybrid-Rechnung: ein optisch lesbares **PDF/A-3** mit **eingebetteter CII-XML**. Der Empfänger kann das PDF wie gewohnt ansehen, während Buchhaltungssysteme die strukturierten Daten direkt auslesen. Die XML entspricht inhaltlich der XRechnung. Zusätzliche Anhänge (z. B. Lieferschein, Leistungsnachweis) können in das PDF/A-3 eingebettet werden.

Peppol / UBL

Für den internationalen Versand über das **Peppol**-Netzwerk erzeugt Factora **UBL** (Universal Business Language) nach **Peppol BIS Billing 3.0**. Korrekturrechnungen werden im UBL-Format als **CreditNote** abgebildet.

Validierung & amtliche Prüfung (KoSIT)

Jede finalisierte Rechnung wird vor der Auslieferung gegen das amtliche Regelwerk geprüft. Factora nutzt dafür den offiziellen **KoSIT-Validator** (Koordinierungsstelle für IT-Standards) — denselben Prüfdienst, den auch die öffentliche Verwaltung einsetzt.

Format / Fall	Status
XRechnung 3.0 (CII)	Amtlich geprüft (KoSIT)
Peppol/UBL Standardrechnung (Billing 3.0)	Amtlich geprüft (Peppol-Validator)
Reverse Charge §13b (AE), inländisch — CII & UBL	Amtlich geprüft (KoSIT)
Self-Billing (389) — CII / XRechnung	Amtlich geprüft (KoSIT)
Self-Billing (389) — Peppol/UBL	Technisch vorbereitet — korrektes Peppol-Self-Billing-Profil wird erzeugt; amtliche Abnahme über einen Peppol-Self-Billing-Validator steht aus

Ergebnisse der Prüfung fließen in die Antwort-Hülle der API ein: KoSIT-Befunde erscheinen als strukturierte Fehler mit Regel (**rule**), Geschäftsbegriff (**bt**) und Fundstelle (**location**).

Format-Auswahl über die API

- Der **Atomic-Endpunkt** liefert die finalisierte Rechnung direkt als **ZUGFeRD-PDF** und **XRechnung-XML** (beides Base64) zurück.
- Zu einer gespeicherten Rechnung lassen sich die XML-Varianten gezielt abrufen: **CII/XRechnung** und **UBL/Peppol**.
- Der **Convert-Endpunkt** rendert EN-16931-Daten ohne Speicherung in geprüftes XML — nützlich zum Testen der eigenen Datenabbildung.

Details siehe Endpoints.

Steuerlogik

Factora bildet die umsatzsteuerliche Behandlung einer Rechnung normkonform nach EN 16931 ab. Jede Position trägt eine **Steuerkategorie (BT-151, UNTDID 5305)** und einen **Steuersatz (BT-152)**; daraus ergeben sich Pflichtangaben wie Befreiungsgründe und die dokumentweite Steueraufstellung.

Steuerkategorien

Code	Bezeichnung	Satz	Anwendung	Befreiungsgrund nötig
S	Regelsteuersatz	19 % oder 7 %	Normale Inlandsumsätze	nein
AA	Ermäßigter Satz	7 %	Ermäßigt besteuerte Leistungen	nein
Z	Nullsatz	0 %	Nullsatz-Umsätze	nein
E	Steuerbefreit	0 %	u. a. §19 Kleinunternehmer	ja (Text)
AE	Reverse Charge	0 %	§13b UStG (Steuerschuld beim Empfänger)	ja (auto)
K	Innergemeinschaftliche Lieferung	0 %	§4 Nr. 1b UStG (EU → EU)	ja (auto)
G	Ausfuhrlieferung	0 %	§4 Nr. 1a UStG (Export Drittland)	ja (auto)
O	Nicht steuerbar	0 %	§3a Abs. 2 UStG (Leistung außerhalb)	individuell

Automatische Ableitung

Wird keine Kategorie ausdrücklich gesetzt, leitet Factora sie ab (erste zutreffende Regel gewinnt):

1. **Ausdrückliche Vorgabe** (z. B. **AE**, **K**, **G**) → wird übernommen.
2. **Kleinunternehmer-Profil** → **E**.
3. **Steuersatz > 0** → **S**.
4. **Steuersatz = 0** → **Z**.

Steuersätze

Unterstützt werden die deutschen Sätze **19 %**, **7 %** und **0 %**. Der Satz wird je Position im Feld **vat_rate** als Prozentwert übergeben (**19.00**, **7.00**, **0.00**).

Befreiungsgründe (BT-120 / BT-121)

Für befreite bzw. steuerfreie Kategorien sind ein **Befreiungstext (BT-120)** und ggf. ein **VATEX-Code (BT-121)** erforderlich. Factora füllt diese bei den deterministischen Fällen automatisch:

Kategorie	Befreiungstext (BT-120)	VATEX-Code (BT-121)
AE	„Steuerschuldnerschaft des Leistungsempfängers (§13b UStG / Art. 196 MwStSystRL)“	VATEX-EU-AE
K	„Innergemeinschaftliche Lieferung (§4 Nr. 1b UStG)“	VATEX-EU-IC
G	„Steuerfreie Ausfuhrlieferung (§4 Nr. 1a UStG)“	VATEX-EU-G
E (§19)	„Kleinunternehmer gemäß §19 UStG“	(<i>leer</i>)
O	individuell zu setzen	(<i>leer</i>)

Eigene Texte/Codes können je Position übergeben werden und haben Vorrang vor den Voreinstellungen.

Reverse Charge §13b (AE)

Die Steuerschuld geht auf den Leistungsempfänger über (z. B. Bauleistungen zwischen Unternehmern). Factora unterstützt **inländische und grenzüberschreitende** Reverse-Charge-Fälle.

Pflichtprüfungen:

- Käufer muss eine **USt-IdNr.** besitzen (BR-AE-02).
- Nur für **B2B/B2G**, nicht für Endverbraucher (B2C).
- Steuersatz muss **0 %** sein.
- Befreiungstext (BT-120) ist Pflicht (wird automatisch gesetzt).

Innergemeinschaftliche Lieferung (K) — §4 Nr. 1b

EU-grenzüberschreitende Lieferung an einen Unternehmer mit USt-IdNr.

Pflichtprüfungen:

- Käufer in einem **EU-Mitgliedstaat** (nicht im Inland).
- Käufer-**USt-IdNr.** vorhanden.
- **VIES-Prüfung** der USt-IdNr.: ungültig → blockiert; nicht erreichbar → nicht-blockierende Warnung mit späterer Wiederholung.
- Hinweis auf fehlenden **Gelangensnachweis** (§17a UStDV) als Warnung.

Ausfuhrlieferung (G) — §4 Nr. 1a

Lieferung an einen Empfänger **außerhalb der EU** (Drittland). Steuersatz 0 %, Befreiungstext automatisch.

Nicht steuerbar (O) — §3a Abs. 2

Leistung mit Ort außerhalb des deutschen Steuergebiets, an einen Empfänger im Drittland. Befreiungstext individuell.

Kleinunternehmer §19 UStG (E)

Bei Kleinunternehmer-Profil:

- Steuersatz **0 %**, Netto = Brutto (keine USt ausgewiesen).
- Kategorie **E**, Befreiungstext „Kleinunternehmer gemäß §19 UStG" ist Pflicht.
- VATEX-Code (BT-121) muss **leer** bleiben.
- Dokument-Hinweis: „Gemäß §19 UStG wird keine Umsatzsteuer berechnet."

Steuerlicher Vertreter (BG-11/12)

Tritt der Verkäufer über einen **Fiskalvertreter** auf (z. B. ausländischer Verkäufer mit Vertreter in DE), trägt die Rechnung dessen Angaben (BT-62 bis BT-69). Wird der Name gesetzt, werden USt-IdNr. und Land zu Pflichtfeldern. Die Angaben können tenantweit hinterlegt oder je API-Rechnung übergeben werden.

Grenzüberschreitende Einordnung

Aus Verkäufer-/Käuferland, USt-IdNr. und Kundentyp leitet Factora die zutreffende Behandlung ab:

- **Gleiches Land** → **S** (Regelsteuer).
- **EU-grenzüberschreitend, B2B mit USt-IdNr.** → **AE** (Reverse Charge).
- **EU-grenzüberschreitend, B2C** → **S** (Satz des Verkäufers).
- **Drittland-Export** → **G**.
- **Nordirland (XI, Brexit-Protokoll)**: Warenlieferungen EU-ähnlich (AE/S), Dienstleistungen als Export (**G**).

Dokumentweite Steueraufstellung (BG-23)

Zusätzlich zur positionsbezogenen Steuer (BG-27/28) erzeugt Factora die zusammengefasste **Steueraufstellung je Satz und Kategorie** (BG-23). Die Summen werden aus den Positionen exakt (Dezimal, keine Float-Rundung) berechnet und gegen die mitgelieferten Kontrollsummen (**validation_totals**) geprüft.

Prüfregeln — was eine Rechnung blockiert

Folgende Steuer-Prüfungen verhindern die Finalisierung (HTTP 400):

Feldname	Kategorie	Bedingung
K_VAT_ID_INVALID	K	Käufer ohne USt-IdNr.
K_COUNTRY_NOT_EU	K	Käufer nicht in der EU
K_COUNTRY_DOMESTIC	K	Käufer im selben Land wie Verkäufer
K_VAT_ID_INVALID	K	USt-IdNr. hat VIES ungültig
AE_CUSTORDER_TYPE_B2C	AE	Reverse Charge an Endverbraucher
AE_VAT_ID_INVALID	AE	Käufer ohne USt-IdNr. (B2B-AE-02)
AE_BT121_INVALID	AE	Befreiungstext b06 (B2B-AE-10)
G_COUNTRY_NOT_TU200	G	Käufer innerhalb der EU
O_COUNTRY_NOT_TU200	O	Käufer innerhalb der EU
E_KU_BT121_INVALID	E	Kleinunternehmer ohne Befreiungstext
E_KU_BT121_SET	E	Kleinunternehmer mit gesetztem VATEX-Code

Nicht-blockierende Warnungen (HTTP 200, protokolliert):

Warncode	Kategorie	Bedingung
K_VAT_ID_UNCHECKED	K	VIES-Dienst nicht erreichbar
K_DELIVERY_PROOF_MISSING	K	Gelangensnachweis noch nicht erfasst

Einfrieren bei Finalisierung (GoBD)

Steuerkategorie, Satz und Befreiungsgründe werden bei der Finalisierung als Snapshot **eingefroren**, damit XML und PDF jederzeit reproduzierbar bleiben. Siehe Compliance & Archivierung.

Compliance & Archivierung

Factora ist auf die deutschen Aufbewahrungs- und Nachvollziehbarkeitsanforderungen (GoBD) ausgelegt. Finalisierte Rechnungen sind unveränderlich, werden verschlüsselt archiviert und jede Statusänderung wird protokolliert.

Unveränderlichkeit finalisierter Rechnungen (GoBD)

Sobald eine Rechnung finalisiert ist, sind ihre rechnungsrelevanten Daten gesperrt. Geschützte (eingefrorene) Felder sind u. a.:

- Rechnungsnummer, Rechnungsdatum, Fälligkeitsdatum
- Käufer/Empfänger-Snapshot
- Netto-, Steuer- und Bruttobeträge
- Leitweg-ID (B2G)
- Steuerkategorie, Steuersatz, Befreiungsgründe je Position

Ein Schreibversuch auf eine finalisierte Rechnung wird abgelehnt. Erlaubt sind nur definierte **Statusübergänge** (z. B. finalisiert → versendet → bezahlt → storniert).

Löschen ist ausschließlich im Entwurfsstatus möglich. Eine finalisierte Rechnung kann nicht gelöscht, sondern nur über eine Korrekturrechnung berichtigt werden.

Verschlüsseltes Archiv

Bei der Finalisierung wird die Rechnung in einem verschlüsselten Archiv abgelegt. Dabei werden die Metadaten (Nummer, Datum, Empfänger, Positionen, Beträge) erfasst und über einen **SHA-256-Integritätshash** gesichert, sodass nachträgliche Veränderungen erkennbar wären. Der Archivstatus jeder Rechnung ist nachvollziehbar (ausstehend / archiviert / fehlgeschlagen).

Änderungsprotokoll (Audit-Log)

Jede Statusänderung einer Rechnung — insbesondere der Übergang vom Entwurf zur finalisierten Rechnung — wird mit Zeitpunkt und auslösendem Akteur in einem Änderungsprotokoll festgehalten.

Normprüfung (EN 16931 / KoSIT)

Vor der Auslieferung durchläuft jede Rechnung die fachliche Prüfung gegen das EN-16931-Regelwerk und — je nach Format — den amtlichen **KoSIT-Validator**. Befunde werden strukturiert zurückgegeben (Regel, Geschäftsbegriff, Fundstelle). Welche Formate amtlich abgenommen sind, steht unter Rechnungsformate.

Datenschutz & Betrieb

- Betrieb und Datenhaltung in **Deutschland**.
- **DSGVO**-konform mit Auftragsverarbeitungsvertrag (AVV).
- Tägliche Sicherung der Daten.
- Sensible Felder (z. B. Bankverbindung) werden verschlüsselt gespeichert.

Datenexport

- Vollständiger Konto-Datenexport auf Anforderung.
- **DATEV-Export** der Buchungsdaten für die Finanzbuchhaltung — siehe Exporte.

API-Überblick

Die öffentliche **Factora API v1** ist eine REST-Schnittstelle zum Erzeugen, Prüfen, Abrufen und Versenden elektronischer Rechnungen aus einem Fremdsystem heraus.

- **Basis-URL:** <https://app.factora.software/api/v1>
- **Datenformat:** JSON (UTF-8)
- **Spezifikation:** OpenAPI 3.1 — interaktiv unter <https://app.factora.software/api/docs/>

Authentifizierung

Jeder Aufruf wird mit einem **API-Schlüssel** als Bearer-Token autorisiert:

```
Authorization: Bearer fa_live_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

API-Schlüssel

- **Live-Schlüssel** beginnen mit `fa_live_`, **Sandbox-Schlüssel** mit `fa_test_`.
- Schlüssel werden **nur als Hash** gespeichert und der Klartext **nur einmal** bei der Erstellung angezeigt.
- Optional pro Schlüssel: **Ablaufdatum**, **IP-Whitelist** (leer = überall gültig) und **Scopes** (leer = alle Rechte).
- Schlüssel lassen sich **rotieren** (neuen erzeugen, alten mit Übergangsfrist auslaufen lassen), deaktivieren oder löschen. Verwaltung erfolgt im Portal.

Scopes (Berechtigungen)

Ein Schlüssel kann auf bestimmte Rechte beschränkt werden. Ohne Einschränkung gelten alle Rechte.

Scope	Bedeutung
<code>invoices:read</code>	Rechnungen lesen, PDF/XML herunterladen
<code>invoices:write</code>	Rechnungen erstellen, finalisieren, versenden
<code>customers:read</code>	Kunden lesen
<code>customers:write</code>	Kunden anlegen
<code>mandanten:read</code>	Mandanten lesen
<code>mandanten:write</code>	Mandanten anlegen/ändern/löschen
<code>exports:read</code>	DATEV-Export herunterladen
<code>webhooks:read</code>	Webhooks und Zustellungen lesen
<code>webhooks:write</code>	Webhooks verwalten, Test senden

Sandbox vs. Live

Mit einem **Sandbox-Schlüssel** lässt sich die Anbindung kostenlos und folgenlos testen. Es durchläuft dieselbe Pipeline wie Live, aber nichts wird gespeichert.

Aspekt	Sandbox	Live
Speicherung	keine (Trockenlauf)	vollständig
Erreichbare Endpunkte	nur Atomic + Hilfsendpunkte (Preview/Convert)	alle
Limits/Kontingent	übersprungen	angewendet (402 bei Limit, 409 bei Dublette)
Idempotenz	optional	erforderlich
Antwort-Status	200 mit <code>meta.sandbox = true</code>	201 bei Anlage

Idempotenz

Schreibende Aufrufe (Atomic Invoice) erfordern im Live-Betrieb einen Idempotenz-Schlüssel, damit ein versehentlich doppelt gesendeter Aufruf nicht zwei Rechnungen erzeugt:

Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000

- **Format:** UUID.
- **Gültigkeit:** 48 Stunden.
- **Wiederholung:** Ein erneuter Aufruf mit demselben Schlüssel liefert die zwischengespeicherte Antwort zurück, gekennzeichnet mit dem Header `Idempotent-Replayed: true`.
- Im Sandbox-Modus ist der Schlüssel optional (es wird ohnehin nichts gespeichert).

Limits (Throttling)

Die API ist pro Schlüssel auf ein Tageskontingent begrenzt. Wird es überschritten, antwortet die API mit **HTTP 429** und einem **Retry-After**-Header. Das Kontingent richtet sich nach dem gebuchten Tarif.

Fehler- und Antwortstruktur

Alle Antworten verwenden eine einheitliche Hülle:

```
{
  "valid": true,
  "data": { },
  "errors": [],
  "meta": {}
}
```

Im Fehlerfall:

```
{
  "valid": false,
  "data": null,
  "errors": [
    {
      "code": "schema",
      "severity": "error",
      "message": "Beschreibung des Fehlers",
      "field": "items[0].vat_rate",
      "rule": "BR-CO-3",
      "bt": "BT-152",
      "location": "//ram:ApplicableTradeTax"
    }
  ],
  "meta": {}
}
```

Das **Fehlerobjekt** enthält **code** und **message** immer, die übrigen Felder je nach Fehlerart: **severity**, **field** (Eingabepfad), **rule** (EN-16931-/Geschäftsregel), **bt** (Geschäftsbegriff) und **location** (XPath bei XML-Befunden).

Fehlercodes

HTTP	Code	Bedeutung
400	schema	Ungültiges JSON, fehlende/falsche Felder, Summen-Abweichung
400	business	Verstoß gegen EN-16931-/GoBD-Geschäftsregel
400	kosit	Befund der amtlichen KoSIT-Prüfung
400	invalid_state	Rechnung im falschen Status (z. B. Versand eines Entwurfs)
400	unsupported_format	Eingabeformat nicht unterstützt
401	authentication	Schlüssel fehlt/ungültig/abgelaufen, IP nicht erlaubt
402	limit_reached	Rechnungslimit des Tarifs erreicht
403	permission	Kein API-Zugang / Sandbox-Schlüssel auf unzulässigem Endpunkt
404	not_found	Objekt existiert nicht oder gehört zu anderem Konto
409	conflict	Doppelte Rechnungsnummer, Dublette, Mandant in Verwendung
429	rate_limited	Tageskontingent überschritten
502	bad_gateway	Webhook-Zustellung fehlgeschlagen (Test)

Mandanten (Multi-Mandanten-Modell)

Ein Integrator kann unter einem Konto beliebig viele **Mandanten** (eigene Endkunden) führen. Jeder Mandant hat eigene Verkäufer-Stammdaten. Ein Aufruf wählt den Mandanten über `mandant_id` im Rechnungs-Payload; dessen Stammdaten werden automatisch als Verkäufer eingesetzt. Kunden und Rechnungen werden **pro Mandant getrennt** geführt. Siehe Endpoints und Atomic Invoice.

Endpoints

Alle Pfade relativ zur Basis-URL <https://app.factora.software/api/v1>. Authentifizierung, Antwort-Hülle und Fehlercodes siehe API-Überblick.

Vollständige Endpunktliste

Methode	Pfad	Zweck	Scope	Sandbox
POST	/invoices/atomic/	Rechnung in einem Aufruf erstellen, finalisieren, PDF+XML zurückgeben	invoices:write	✓
POST	/invoices/preview/	Summen live berechnen (ohne Speicherung)	invoices:read	✓
POST	/invoices/convert/	EN-16931-JSON → geprüftes XML (ohne Speicherung)	invoices:write	✓
GET	/invoices/	Rechnungen auflisten (paginiert, filterbar)	invoices:read	—
POST	/invoices/	Rechnung als Entwurf anlegen	invoices:write	—
GET	/invoices/{id}/	Einzelne Rechnung abrufen	invoices:read	—
POST	/invoices/{id}/finalize/	Entwurf finalisieren (PDF/XML erzeugen)	invoices:write	—
POST	/invoices/{id}/send/	Finalisierte Rechnung per E-Mail versenden	invoices:write	—
GET	/invoices/{id}/pdf/	ZUGFeRD-PDF herunterladen	invoices:read	—
GET	/invoices/{id}/xml/	XRechnung-XML herunterladen	invoices:read	—
GET	/invoices/{id}/xml/CLIX/	CLIX-Rechnung-XML live erzeugen	invoices:read	—
GET	/invoices/{id}/xml/UBL/	UBL/Peppol-XML live erzeugen	invoices:read	—
POST	/invoices/{id}/dispatch/	Rechnung zum Transport einreihen (E-Mail/Peppol)	invoices:write	—
GET	/invoices/{id}/dispatch-history/	Verseilungshistorie abrufen	invoices:read	—
GET	/customers/	Kunden auflisten	customers:read	—
POST	/customers/	Kunden anlegen	customers:write	—
GET	/mandanten/	Mandanten auflisten	mandanten:read	—
POST	/mandanten/	Mandant anlegen	mandanten:write	—
GET	/mandanten/{id}/	Mandant abrufen	mandanten:read	—
PATCH	/mandanten/{id}/	Mandant ändern (partiell)	mandanten:write	—
DELETE	/mandanten/{id}/	Mandant löschen (nur ohne Rechnungen/Kunden)	mandanten:write	—
POST	/mandanten/bulk/	Bis zu 200 Mandanten anlegen (alles-oder-nichts)	mandanten:write	—
GET	/exports/datev/	DATEV-Buchungsdaten als CSV	exports:read	—
GET	/webhooks/	Webhooks auflisten	webhooks:read	—
POST	/webhooks/	Webhook anlegen	webhooks:write	—
GET	/webhooks/{id}/	Webhook abrufen	webhooks:read	—
PATCH	/webhooks/{id}/	Webhook ändern	webhooks:write	—
DELETE	/webhooks/{id}/	Webhook deaktivieren	webhooks:write	—
POST	/webhooks/{id}/test/	Test-Webhook senden	webhooks:write	—
GET	/webhooks/{id}/deliveries/	Liste der Zustellungen eines Webhooks auflisten	webhooks:read	—

Rechnungen

Atomic Invoice — **POST** `/invoices/atomic/`

Der zentrale Endpunkt für die ERP-Integration: ein einziger Aufruf erstellt, finalisiert und prüft die Rechnung und liefert das fertige **ZUGFeRD-PDF** und die **XRechnung-XML** (beide Base64) zurück. Erfordert im Live-Betrieb einen Idempotenz-Schlüssel. Das **vollständige Feldschema** steht unter Atomic Invoice.

Preview — **POST** `/invoices/preview/`

Berechnet aus übergebenen Positionen die Summen exakt (Dezimal) inklusive Aufschlüsselung je Steuersatz. Keine Speicherung, auch mit Sandbox-Schlüssel und bei inaktivem Abo nutzbar.

Convert — **POST** `/invoices/convert/`

Rendert EN-16931-Rechnungsdaten ohne Speicherung in geprüftes XML (CII/XRechnung oder UBL, je nach Rechnungstyp) und gibt KoSIT-Befunde zurück. Ideal, um die eigene Datenabbildung zu testen.

Klassischer Lebenszyklus

Statt des Atomic-Wegs lässt sich eine Rechnung auch schrittweise führen: anlegen (**POST** `/invoices/`) → finalisieren (`/finalize/`) → herunterladen (`/pdf/`, `/xml/`) → versenden (`/send/`) oder einreihen (`/dispatch/`). Listen sind paginiert und filterbar; per **mandant**-Filter lässt sich auf einen Mandanten einschränken.

Mandanten

CRUD über `/mandanten/`. Der **Bulk-Endpunkt** legt bis zu 200 Mandanten in einer Transaktion an (**alles oder nichts** — schlägt eine Zeile fehl, z. B. durch einen doppelten Namen, wird nichts angelegt; Antwort 409). Ein Mandant kann nicht gelöscht werden, solange Rechnungen oder Kunden an ihm hängen (409, GoBD).

Kunden

Auflisten und Anlegen über `/customers/`. Auf dem Mandanten-Pfad werden Kunden pro Mandant getrennt geführt.

Exporte

GET `/exports/datev/` — siehe Exporte.

Webhooks

CRUD, Testversand und Zustellhistorie über `/webhooks/` — siehe Webhooks.

Atomic Invoice — vollständiges Feldschema

POST /api/v1/invoices/atomic/

Erstellt, finalisiert und prüft eine Rechnung in einem einzigen Aufruf und liefert das fertige ZUGFeRD-PDF und die XRechnung-XML zurück. Dieses Dokument listet **jedes Feld** des Payloads mit Typ, Pflicht/Optional, Default, Format und EN-16931-Bezug (BT-Codes).

Konventionen: Beträge sind Dezimal-Strings (2 Nachkommastellen), Mengen 3 Stellen, Sätze 2 Stellen; Datumsangaben ISO 8601 (YYYY-MM-DD).

Aufbau des Payloads

Feld	Typ	Pflicht	Default	Bemerkung
api_mode	string	ja	—	muss "atomic_single_post" sein
invoice_header	Objekt	ja	—	Rechnungskopf (s. u.)
validation_totals	Objekt	ja	—	Kontrollsummen aus dem Quellsystem
seller_snapshot	Objekt	bedingt*	—	Verkäufer-Stammdaten
mandant_id	integer	bedingt*	—	alternativ: Mandant referenzieren
buyer	Objekt	ja	—	Käufer/Empfänger
shipping	Objekt	nein	—	abweichende Lieferadresse
items	Array	ja	—	min. 1 Position
attachments	Array	nein	[]	bis 10 Anhänge, max. 15 MB gesamt
extended_fields	Objekt	nein	{}	optionale EN-16931-Zusatzfelder

* **Genau eines** von `seller_snapshot` oder `mandant_id` ist anzugeben. Bei `mandant_id` werden die Verkäuferdaten automatisch aus dem Mandanten gesetzt.

Rechnungskopf — invoice_header



Kontrollsummen — validation_totals

Feld	Typ	Pflicht	Bemerkung
net_amount	decimal	ja	Summe der Netto-Positionsbeträge
vat_amount	decimal	ja	Gesamte USt
gross_amount	decimal	ja	Brutto (Netto + USt)

Der Server berechnet die Summen aus den Positionen neu und vergleicht sie mit diesen Werten; eine Abweichung führt zu HTTP 400.

Verkäufer — seller_snapshot

Feld	Typ	Pflicht	Default	BT	Bemerkung
name	string(255)	ja	—	BT-27	Firmenname
street	string(255)	ja	—	BT-31	Straße
zip	string(20)	ja	—	BT-34	PLZ
city	string(255)	ja	—	BT-35	Ort
country	string(2)	nein	"DE"	BT-40	ISO 3166-1
country_subdivision	string(100)	nein	""	BT-39	Bundesland/Region
address_line2	string(255)	nein	""	BT-36	Adresszusatz
address_line3	string(255)	nein	""	BT-162	Adresszusatz
vat_id	string	nein	""	BT-31	USt-IdNr.
tax_number	string	nein	""	—	Steuernummer
gln	string	nein	""	—	GLN (13 Stellen, GS1-Prüfziffer)
identifizier	string(64)	nein	""	BT-29	weitere Verkäufererkennung
trading_name	string(255)	nein	""	BT-28	Handelsname
legal_registration_id	string(64)	nein	""	BT-30	Handelsregisternummer
additional_legal_info	string(255)	nein	""	BT-33	rechtliche Zusatzinfo
iban	string	nein	""	BT-84	Zahlungs-IBAN
bic	string	nein	""	—	BIC/SWIFT
contact	Objekt	nein	—	BG-6	Ansprechpartner (s. u.)
tax_representation	Objekt	nein	—	BG-11/12	steuerlicher Vertreter (s. u.)

Ansprechpartner — contact

Feld	Typ	Pflicht	Bemerkung
name	string(255)	nein	Kontaktperson
phone	string(50)	nein	Telefon
email	string(255)	nein	E-Mail

Steuerlicher Vertreter — tax_representative (BG-11/12)

Alle Felder optional; wird **name** gesetzt, werden **vat_id** und **country** zu Pflichtangaben.

Feld	Typ	BT	Bemerkung
name	string(255)	BT-62	Name des Vertreters
vat_id	string(64)	BT-63	USt-IdNr. des Vertreters
street	string(255)	BT-64	Straße
address_line2	string(255)	BT-65	Adresszusatz
address_line3	string(255)	BT-164	Adresszusatz
zip	string(20)	BT-67	PLZ
city	string(255)	BT-66	Ort
country_subdivision	string(100)	BT-68	Region
country	string(2)	BT-69	Land

Käufer — buyer

Feld	Typ	Pflicht	Default	BT	Bemerkung
name	string(255)	ja	—	BT-44	Firmenname
street	string(255)	ja	—	BT-49	Straße
zip	string(20)	ja	—	BT-50	PLZ
city	string(255)	ja	—	BT-50	Ort
country	string(2)	nein	"DE"	BT-55	ISO 3166-1
country_subdivision	string(100)	nein	" "	BT-54	Bundesland/Region
address_line2	string(255)	nein	" "	BT-51	Adresszusatz
address_line3	string(255)	nein	" "	BT-163	Adresszusatz
vat_id	string	nein	" "	BT-48	USt-IdNr. (Pflicht bei AE/K)
gln	string	nein	" "	BT-44	GLN (13 Stellen, GS1-Prüfziffer)
identifizier	string(64)	nein	" "	BT-46	weitere Käuferkennung
debtor_number	string(20)	nein	" "	—	DATEV-Debitorennummer (nur Export, nicht im XML)
branch_id	string	nein	" "	—	Filial-/Standortkennung
trading_name	string(255)	nein	" "	BT-45	Handelsname
legal_registration_id	string(64)	nein	" "	BT-47	Handelsregisternummer
contact	Objekt	nein	—	BG-9	Ansprechpartner (wie oben)

Lieferadresse — shipping (BG-13/15)

Optional; fehlt sie, gilt die Käuferadresse. Wird sie angegeben, sind **name**, **street**, **zip**, **city** Pflicht.

Feld	Typ	Pflicht	Default	BT
name	string(255)	ja	—	BT-71
street	string(255)	ja	—	BT-73
zip	string(20)	ja	—	BT-74
city	string(255)	ja	—	BT-75
country	string(2)	nein	"DE"	BT-80
country_subdivision	string(100)	nein	" "	BT-79
address_line2	string(255)	nein	" "	BT-76
address_line3	string(255)	nein	" "	BT-165
gln	string	nein	" "	—

Positionen — items[]

Pflichtfelder je Position: **description**, **quantity**, **unit**, **unit_price_net**, **vat_rate**.

Feld	Typ	Pflicht	Default	BT	Bemerkung
description	string(500)	ja	—	BT-154	Bezeichnung
quantity	decimal(...3)	ja	—	BT-129	Menge
unit	string(10)	ja	—	BT-130	Einheit (UNCL 572: HUR Stunde, KGW kg, CSD Stück, ...)
unit_price_net	decimal	ja	—	BT-146	Nettoeinzelpreis
vat_rate	decimal(5,2)	ja	—	BT-152	Steuersatz in % (19.00 / 7.00 / 0.00)
tax_category_code	string	nein	**	BT-151	Kategorie (SAAZ(EINZIG?)) — siehe Steuerlogik
tax_exemption_reason_code	string	nein	**	BT-121	VATEX-Code (Pflicht bei Befreiung)
tax_exemption_reason_text	string	nein	**	BT-120	Befreiungstext
sku	string(50)	nein	**	BT-155	Artikelnummer Verkäufer
gtin	string	nein	**	BT-157	GTIN (13/14 Stellen, GS1-Prüfziffer)
buyer_item_id	string(100)	nein	**	BT-156	Artikelnummer Käufer
discount_percent	decimal(5,2)	nein	—	BT-139	Rabatt in %
discount_amount	decimal	nein	—	BT-138	Rabatt absolut
discount_base_amount	decimal	nein	—	BT-137	Rabatt-Basis
discount_reason	string	nein	**	—	Rabattgrund (Freibetrag)
discount_reason_code	string(16)	nein	**	BT-140	Rabattgrund-Code (UNTDID 5189)
charge_amount	decimal	nein	—	BT-141	Zuschlag absolut (BG-218)
charge_base_amount	decimal	nein	—	BT-142	Zuschlag-Basis
charge_percent	decimal(5,2)	nein	—	BT-143	Zuschlag in %
charge_reason	string(255)	nein	**	BT-144	Zuschlaggrund
charge_reason_code	string(16)	nein	**	BT-145	Zuschlaggrund-Code (UNTDID 7161)
period_start	date	nein	—	BT-134	Leistungszeitraum Beginn
period_end	date	nein	—	BT-135	Leistungszeitraum Ende
line_note	string(1000)	nein	**	BT-127	Positionshinweis
object_identifier	string(200)	nein	**	BT-128	Objektkennung
order_line_reference	string(50)	nein	**	BT-132	Bestellpositions-Referenz
line_accounting	string(200)	nein	**	BT-133	Kostenstelle (Position)
item_description	string(1000)	nein	**	BT-154	erweiterte Beschreibung
classification	string(100)	nein	**	BT-158	Klassifizierung (z. B. eCIGs)
classification_code	string(10)	nein	**	BT-159-1	Klassifizierungsschema (UNCL 7143)
country_of_origin	string(2)	nein	**	BT-159	Ursprungsland
attribute_name	string(200)	nein	**	BT-160	Merkmalsname
attribute_value	string(200)	nein	**	BT-161	Merkmalswert
gross_price	decimal	nein	—	BT-148	Bruttoeinzelpreis vor Positionsrabatt
price_discount	decimal	nein	—	BT-147	Preisnachlass (Brutto - Netto)
price_base_quantity	decimal(...3)	nein	—	BT-149	Preisbasis-Menge
price_base_unit	string(10)	nein	**	BT-150	Einheit der Preisbasis

Anhänge — attachments[]

Bis zu **10 eingebettete Anhänge**, **max. 15 MB** je Datei und **gesamt**. Pro Anhang ist **entweder** `content_base64` **oder** `external_url` anzugeben. Der MIME-Typ wird gegen eine Whitelist geprüft und der Dateiinhalt zusätzlich anhand der „Magic Bytes“ verifiziert.

Erlaubte MIME-Typen: PDF, PNG, JPEG, CSV, ODS (OpenDocument), XLSX, XLS.

Feld	Typ	Pflicht	Default	BT	Bemerkung
<code>filename</code>	string(255)	ja	—	—	Dateiname
<code>mime_type</code>	string(100)	ja	—	BT-125-1	aus Whitelist
<code>type</code>	string	nein	"OTHER"	—	INVOICE_IMAGE / DELIVERY_NOTE / OTHER
<code>content_base64</code>	string	bedingt	""	BT-125	eingebettete Datei (XOR <code>external_url</code>)
<code>external_url</code>	string(500)	bedingt	""	BT-124	externer Link (XOR <code>content_base64</code>)
<code>document_reference</code>	string(200)	nein	""	BT-122	Dokumentkennung
<code>description</code>	string(255)	nein	""	BT-123	Beschreibung
<code>delivery_note_number</code>	string	nein	""	—	bei <code>type=DELIVERY_NOTE</code>

Zusatzfelder — extended_fields

Frei verwendbares Objekt für weitere EN-16931-Felder. Werte aus dem Aufruf überschreiben etwaige am API-Schlüssel hinterlegte Voreinstellungen.

Antwort

Erfolg (Live): HTTP 201. **Sandbox:** HTTP 200 mit `meta.sandbox = true`.

```
{
  "valid": true,
  "data": {
    "id": 12345,
    "invoice_number": "RE-2026-001",
    "status": "final",
    "total": "1190.00",
    "pdf_base64": "JVBERi0xLjQK...",
    "xml_base64": "PD94bWwgdmVy..."
  },
  "errors": [],
  "meta": {}
}
```



Fehlerfälle und Fehlercodes: siehe API-Überblick.

Wichtige Kombinationsregeln

1. Genau eines von `seller_snapshot` / `mandant_id`.
2. Bei befreiten Kategorien (E, AE, K, G, O) Befreiungstext/-code angeben (AE/K/G werden automatisch gefüllt).
3. AE/K erfordern eine Käufer-USt-IdNr.; AE nur für B2B/B2G.
4. `validation_totals` müssen zur Positionsberechnung passen.
5. GLN/GTIN, falls angegeben, müssen gültige GS1-Prüfziffern haben.
6. Anhänge: `content_base64` XOR `external_url`.

Webhooks

Webhooks benachrichtigen ein Fremdsystem in Echtzeit über Ereignisse zu Rechnungen. Factora sendet dazu einen signierten HTTP-POST an eine von Ihnen hinterlegte HTTPS-URL.

Verwaltung über die Endpunkte unter `/api/v1/webhooks/` (siehe Endpoints).

Ereignistypen

Ereignis	Auslöser
<code>invoice.created</code>	Rechnungsentwurf angelegt
<code>invoice.finalized</code>	Rechnung finalisiert (PDF/XML erzeugt)
<code>invoice.dispatched</code>	Rechnung zum Transport eingereicht
<code>invoice.delivered</code>	Rechnung erfolgreich zugestellt (E-Mail/Peppol)
<code>invoice.paid</code>	Rechnung als bezahlt markiert
<code>invoice.cancelled</code>	Rechnung storniert
<code>invoice.rejected</code>	Rechnung abgelehnt (Prüf-/Zustellfehler)

Beim Anlegen eines Webhooks wählen Sie die abonnierten Ereignisse. Zum Testen existiert zusätzlich `webhook.test` (siehe unten).

Aufbau der Zustellung

Jeder Webhook-Request enthält:

Header	Wert
<code>Content-Type</code>	<code>application/json</code>
<code>X-Factora-Event</code>	Ereignistyp (z. B. <code>invoice.finalized</code>)
<code>X-Factora-Signature</code>	<code>sha256=<hex></code> — HMAC-SHA256 des Rumpfes
<code>User-Agent</code>	<code>Factora-Webhooks/1.0</code>

Signaturprüfung

Die Signatur sichert Echtheit und Unverändertheit. Berechnung:

```
X-Factora-Signature = "sha256=" + HMAC_SHA256(secret, raw_request_body)
```

Der Empfänger berechnet denselben HMAC mit dem **bei der Webhook-Anlage vereinbarten Secret** über den **rohen Request-Body** und vergleicht das Ergebnis. Stimmen die Werte nicht überein, ist der Request abzulehnen.

Das Secret wird auf Ihrer Seite gewählt und bei Factora verschlüsselt gespeichert. Es verlässt das System nicht im Klartext.

Zustellung & Wiederholung

Eigenschaft	Wert
Erfolgskriterium	HTTP-Status 200–299
Timeout	10 Sekunden je Versuch
Maximale Versuche	5
Wiederholungsabstände	30 s · 2 min · 10 min · 1 h · 6 h
Weiterleitungen	werden nicht gefolgt (Schutz gegen Rebinding)
Automatische Abschaltung	nach mehreren aufeinanderfolgenden Fehlschlägen

Die Zustellhistorie eines Webhooks ist über `GET /webhooks/{id}/deliveries/` abrufbar (Versuche, Status, Antwortzeit, nächste Wiederholung). Jede Zustellung führt u. a. `event_type`, `attempts`, `max_attempts`, `response_status`, `success`, `completed_at`.

Test-Webhook

`POST /webhooks/{id}/test/` sendet **synchron** einen Test-Request an die hinterlegte URL und gibt das Ergebnis direkt zurück:

```
{ "valid": true, "data": { "success": true, "status_code": 200 }, "meta": { "duration_ms": 342 } }
```

Ist die URL nicht erreichbar, antwortet die API mit `success: false` und einer Fehlerbeschreibung (HTTP 502).

Sicherheitsanforderungen an die Ziel-URL

- Nur **HTTPS**.
- Nur **öffentlich erreichbare** Adressen (interne/lokale Hosts werden abgelehnt — SSRF-Schutz).

Exporte

DATEV-Export — GET /api/v1/exports/datev/

Erzeugt die **Buchungsdaten** der finalisierten Rechnungen eines Zeitraums als CSV im DATEV-Format — direkt importierbar in die Finanzbuchhaltung bzw. durch den Steuerberater.

Parameter

Parameter	Pflicht	Format	Bemerkung
<code>date_from</code>	ja	<code>YYYY-MM-DD</code>	Beginn des Zeitraums
<code>date_to</code>	ja	<code>YYYY-MM-DD</code>	Ende des Zeitraums

Erforderlicher Scope: `exports:read`.

Ausgabe

Eigenschaft	Wert
Medientyp	<code>text/csv; charset=windows-1252</code>
Kodierung	Windows-1252 (CP1252) — DATEV-kompatibel
Dateiname	<code>datev_{date_from}_{date_to}.csv</code>
Format	DATEV-Buchungsdaten (Buchungsstapel)
Umfang	alle finalisierten Rechnungen im Zeitraum (Entwürfe ausgeschlossen), kontogebunden je Konto

Jede Rechnung wird als Buchungszeile abgebildet; bei gemischten Steuersätzen erfolgt eine Aufteilung je Satz. Enthalten sind u. a. Betrag, Soll/Haben- Kennzeichen, Konto und Gegenkonto, Buchungsschlüssel (BU), Belegdatum sowie die Debitorennummer (sofern am Käufer hinterlegt).

Fehler

HTTP	Bedingung	Code
400	<code>date_from/date_to</code> fehlt	<code>schema</code>
400	Datum nicht im Format <code>YYYY-MM-DD</code>	<code>schema</code>

Weitere Exporte

- **Rechnungsdokumente:** ZUGFeRD-PDF und XRechnung-/UBL-XML lassen sich pro Rechnung über die Rechnungs-Endpunkte herunterladen (siehe Endpoints).
- **Konto-Datenexport:** vollständiger Export der eigenen Kontodaten auf Anforderung (siehe Compliance & Archivierung).